

30. Distributed Cache (Redis / Memcached-style)

Interviewer: We've got a product-catalog service backed by Postgres. Reads outnumber writes 1000:1, and at peak the database is the bottleneck — CPU pegged, connection pool exhausted, p99 read latency in the hundreds of ms. Design a **distributed caching layer** in front of it that can serve millions of reads/sec out of memory, evict intelligently when it's full, and stay reasonably consistent with the database when a product changes.

Candidate: This is a different flavor of "distributed data" problem than a KV store or a database. A distributed KV store has to be the **source of truth** — durable, never allowed to lose a write. A cache is the opposite contract: it's **disposable, bounded, and allowed to be wrong for a little while**, in exchange for being fast. That changes almost every design decision. Let me clarify first.

1. Clarifying the requirements

Candidate: A few questions before sizing anything: 1. What's the read/write ratio and the acceptable staleness window — is "correct within a few seconds of a write" fine, or do some reads need to see their own write immediately? 2. How big is the working set relative to total data — does the "hot" 10% fit comfortably in a cache cluster's memory, or are we caching almost everything? 3. Is there a known hot-key risk — a viral product, a homepage banner — that gets disproportionate traffic? 4. On a cache **miss**, is it acceptable for that one request to pay a DB round-trip? On a cache **outage**, do we degrade to hitting the DB directly, or fail the request?

Interviewer: A few seconds of staleness is fine for most reads; checkout-adjacent reads need to see their own recent writes. The hot set is maybe 5-10% of the catalog but it shifts (flash sales, trending items). Yes — assume a handful of items get 100x normal traffic sometimes. On a miss, one DB round-trip per key is fine. On a full cache outage, we must degrade gracefully, not take the DB down with us.

Candidate: Good, that's exactly the cache-shaped set of constraints. Requirements:

Functional - `GET(key)` — return cached value if present and unexpired; else fetch from DB, populate cache, return. - `SET/UPDATE(key, value)` — write path keeps cache and DB reasonably in sync. - `DELETE/INVALIDATE(key)` — remove or expire a key when the underlying row changes. - Bounded memory per node — when full, evict something, don't error out or OOM.

Non-functional - **Throughput:** millions of reads/sec, cache-node latency in sub-millisecond to low-ms. -

Bounded staleness: a few seconds typical; certain paths need read-your-writes. - **Graceful**

degradation: losing a cache node — or the whole cache tier — must not collapse the database; it should degrade to *slower*, not *down*. - **Hot-key resilience:** a single popular key must not overwhelm the one shard that owns it. - **This is NOT the KV-store problem.** We're not building a new source of truth with replication quorums and conflict resolution (see doc 11) — the DB is still the source of truth. We reuse **consistent hashing** to spread keys across cache nodes, but everything else here — eviction, TTL,

invalidation, cache-vs-DB consistency, hot keys, stampedes — is specific to the fact that this store is a *cache*, not a database.

2. Estimation

Catalog: 50M products, avg cached object ~2 KB (denormalized JSON) -> 100 GB full dataset.
Hot working set: ~10% actively read -> ~10 GB "must be fast" data, rest is long-tail.

Traffic: 2M reads/sec at peak, 2k writes/sec (typical read-heavy skew).

DB alone: a well-tuned Postgres replica set tops out around 20-50k qps for this workload -> DB can cover AT MOST ~2% of read traffic. The cache must absorb ~98%+.

Target cache hit ratio: >=95%. At 95% hit rate, DB still sees 100k qps of misses+writes -- already close to its ceiling, so hit ratio isn't a nice-to-have, it's the whole game.

Cache node sizing: instances with ~50 GB usable RAM (leaving headroom for overhead) -> 100 GB dataset / 50 GB per node = 2 nodes for CAPACITY, but 2M reads/sec / ~200k ops/sec per node (Redis single-threaded core) ~ 10 nodes for QPS. -> QPS drives node count more than data size does. This is the opposite of the KV-store doc (11), where storage volume drove node count -- here we're memory-latency-bound and single-core-ops-bound per node, not disk-bound.

3. V1 — a single cache node, cache-aside

Candidate: Simplest version: one Redis instance sits in front of Postgres, and the application does the read-through logic itself — this is the **cache-aside** pattern.

```
[ App server ] --GET key--> [ Redis ]
      |
      | hit: return value — fast path, most requests
      |
      | miss: [ App ] --SELECT--> [ Postgres ] --> [ App ] --SET key,val,TTL--> [ Redis ]
      |                                     --> return value to caller
```

Why this dies at our scale: one node caps at ~200k ops/sec and ~50 GB — nowhere near 2M ops/sec or 100 GB. And a single node is a single point of failure: if it dies, every request becomes a cache miss simultaneously and the full 2M reads/sec slams into Postgres — exactly the outage we're trying to prevent. We need to **shard** for throughput+capacity and **replicate** for availability.

4. Scaling out — sharding across a cache cluster

Interviewer: Fine, shard it. How do keys map to nodes, and who does the routing?

Candidate: Same partitioning primitive as the KV-store doc — **consistent hashing** (Redis Cluster uses 16,384 fixed hash slots distributed across nodes, which is a variant of the same idea; Memcached clients

typically use client-side ketama hashing over a ring). I won't re-derive it here — the short version: hash the key, land on a slot/point on a ring, the owning node serves it, and adding/removing a node only reshuffles $\sim 1/N$ of keys instead of all of them. Two ways to route:

```
CLIENT-SIDE SHARDING (Memcached-style, "dumb" server)
[ App ] --hash(key) locally--> knows which node owns it --> talks DIRECTLY to that node
+ one network hop, minimal latency
- every client must embed the hashing/ring logic and stay in sync with cluster topology

PROXY-BASED SHARDING (Redis Cluster / Twemproxy-style, "smart" tier)
[ App ] --GET key--> [ Proxy / cluster-aware client ] --routes--> [ Node_i ]
+ clients stay simple; topology changes handled centrally
- one extra hop (or a smart client library that caches the slot map, splitting the
  difference – this is what Redis Cluster client libraries actually do)
```

Choice: Redis Cluster with cluster-aware client libraries — clients cache the slot map (like the KV-store doc's "client-side ring-aware routing") and talk directly to the right node most of the time, falling back to a redirect (MOVED / ASK) during resharding. This gets the low-hop latency of client-side sharding without hand-rolling ring logic in every service.

5. Replication — surviving a node loss without a cache-clearing event

Interviewer: A shard holding a slice of your hot set dies. What happens to those keys?

Candidate: Without replicas, that shard's entire slice of the keyspace instantly goes cold — every request for those keys becomes a DB read at once, a **localized stampede** (more on stampedes in §15). So each shard gets **1-2 replicas** (Redis replication, async by default):

```
[ Primary shard 3 ] — async replicate —> [ Replica shard 3 ]
      | serves writes + reads                | promotable on failover
      |                                     |
node dies -----> Sentinel/Cluster promotes
                        replica to primary in seconds
```

- **Why async replication, not sync?** A cache's whole value proposition is *speed*. Sync replication (wait for the replica to ack) would add a network round-trip to every write, defeating the purpose. Async means a replica can be a few writes behind on failover — for a cache, that's a **stale value, not a lost transaction**, and the DB is still the source of truth behind it. This is the single biggest philosophical difference from the KV-store doc, where losing an unacked write is a real correctness bug — here, the worst case is "served a slightly old value," which the TTL/invalidation path already tolerates.
 - Failover promotes a replica in seconds; in the gap, requests for that shard's keys miss and fall through to the DB — brief, contained load, not a full-cluster stampede.
-

6. Eviction — the policy question a KV store never has to answer

Interviewer: Your cache cluster is full. A new key needs space. What gets evicted, and why does this question not even exist for a normal database?

Candidate: This is the defining cache-specific problem. A database keeps growing disk forever (or you provision more); a cache has a **fixed memory budget by design** — that's what makes it fast and cheap per byte. When it's full, something *must* be thrown away, and the policy for what to throw away is a first-class design decision:

```
LRU (Least Recently Used) – evict the key not accessed in the longest time.
+ matches "recently viewed products stay hot" intuition; cheap approximations exist
  (Redis uses sampled LRU, not exact, to avoid a global access-order data structure).
- a single big sequential scan (e.g. a batch export job) can flush the whole cache with
  one-time-use keys, evicting genuinely hot data ("cache pollution").

LFU (Least Frequently Used) – evict the key with the lowest access COUNT (with decay over
time so old popularity fades).
+ resistant to the scan-pollution problem above -- a one-off scan's keys have low
  frequency and get evicted first even if "recently" used.
- slower to adapt when a NEW item suddenly goes viral (low historical frequency at first).

TTL (Time To Live) – every key has an expiry; evict/expire regardless of access pattern.
+ bounds staleness directly -- this is how we cap "how wrong can a stale value be."
+ orthogonal to LRU/LFU -- used TOGETHER, not instead of.
- doesn't help with memory pressure by itself if the TTL is long relative to write volume.

RANDOM / FIFO – evict oldest-inserted or a random key.
+ trivial, O(1), no bookkeeping.
- throws away hot data as readily as cold data; rarely the right default.
```

Choice: LFU with decay as the eviction policy, TTL as an independent staleness bound on top, applied per-node when that node hits its **maxmemory**. LRU is the more familiar default and fine for many workloads, but our requirements called out **shifting hot sets and occasional big scans** (catalog exports, admin bulk reads) — exactly the scan-pollution case LFU is built to resist. TTL still caps staleness independently: a product can be "frequently used" and still expire after, say, 5 minutes, so a price update is never wrong for more than that window even without an explicit invalidation.

```
Node at maxmemory, new SET arrives:
-> eviction policy picks a victim key (LFU: lowest decayed frequency)
-> victim evicted, new key inserted
-> this happens continuously under load, not as a "cleanup pass"
```

7. Caching patterns — who writes to the cache, and when

Interviewer: Walk me through how a write actually keeps the cache from lying to readers.

Candidate: Three well-known patterns, each a different answer to "when does the cache get updated relative to the DB write":

CACHE-ASIDE (lazy loading) – app owns both, cache is populated ONLY on read-miss
write: App → UPDATE DB → DELETE/invalidate cache key (don't repopulate yet)
read: App → GET cache → miss → SELECT DB → SET cache → return
+ simplest; cache only ever holds keys that were actually requested (no wasted memory)
+ a cache outage degrades to "every read hits the DB," survivable if the DB can take some load
– the read AFTER a write pays one guaranteed DB round-trip (until repopulated)
– a race: a slow read-miss-populate can overwrite a fresher DELETE from a concurrent write

WRITE-THROUGH – every write goes through the cache, which writes to the DB synchronously
write: App → SET cache → cache synchronously writes DB → ack
read: App → GET cache → always warm for anything ever written
+ cache and DB never diverge on the write path; reads are always warm post-write
– write latency = cache write + DB write, always – no write is ever "just fast"
– the DB still absorbs every single write; this pattern doesn't reduce write load, only read load

WRITE-BACK (write-behind) – write lands in cache, DB write is deferred/async/batched
write: App → SET cache → ack immediately → async worker flushes to DB later (batched)
read: App → GET cache → warm
+ fastest writes; can batch/coalesce many writes into fewer DB ops
– a cache-node crash before flush LOSES writes that were only ever in memory – unacceptable for anything we can't afford to lose, fine for e.g. view counters, ephemeral metrics

Choice: cache-aside for the catalog (read-heavy, DB is authoritative, occasional stale-miss is fine), invalidate-on-write rather than repopulate-on-write. On a product update: `UPDATE postgres SET ... ; DEL cache:product:{id}`. We delete rather than SET the new value directly into the cache from the write path, because deleting is simpler to get right and avoids two writers (the update path and a concurrent read-repopulate) racing to decide the "current" cached value – the next reader just repopulates from the now-updated DB. Write-through would add DB-write latency to every catalog edit for no benefit (writes are rare here); write-back would risk losing an edit that was never flushed – unacceptable for product data, though it's exactly the right call for something like a view-count or like-count where losing a few increments on a crash is a non-issue.

8. Invalidation and cache-vs-DB consistency

Interviewer: You said "DELETE the key on write." What if the delete never reaches every replica, or a stale read sneaks in between the DB commit and the delete?

Candidate: This is where a cache's contract is explicitly **best-effort, TTL-bounded**, not strongly consistent – and that's a deliberate, stated tradeoff, not a bug we're hiding:

```
t0 UPDATE postgres SET price=9.99 WHERE id=42      (DB now has the new price)
t1 DEL cache:product:42                            (cache invalidated)
t2 any GET after t1 → miss → reads DB → caches the NEW price    correct
```

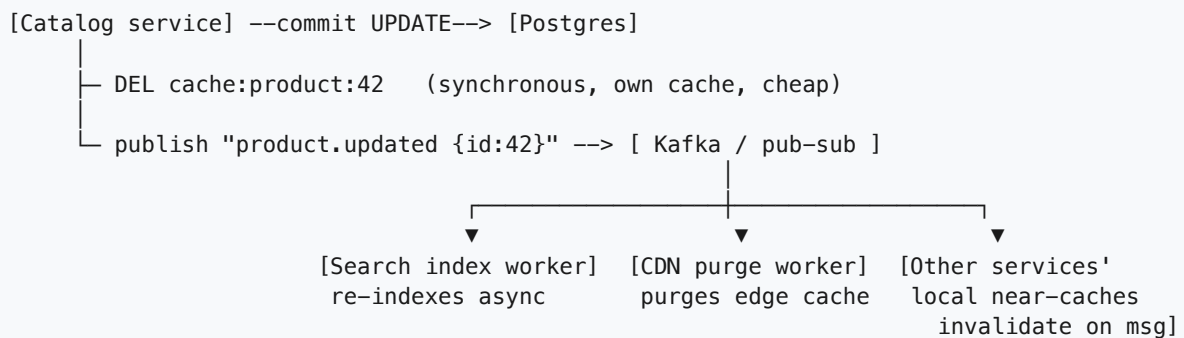
THE RACE: a read that started JUST BEFORE t1 but returns JUST AFTER:
t0.5 GET cache:product:42 → HIT, old price 8.99, returned to a caller ← stale by a hair
t1 DEL happens right after
→ that one caller saw a value that was already outdated by a few ms. Bounded, rare, and healed by TTL if the DEL is ever missed/dropped for any reason.

- **TTL is the safety net, not the primary mechanism.** Even if an invalidation message is lost (network blip, a bug, a replica that didn't get the `DEL`), every key still expires on its own within the TTL — worst case, staleness is bounded by TTL, never unbounded.
- **Read-your-writes for the checkout path** (a requirement from §1): for flows where a user must see their own write immediately, bypass the cache on the next read for that entity (or route it to the primary DB directly) rather than trusting cache invalidation to have propagated in time. This is the same reasoning as "client-side session stickiness" — solve it narrowly for the paths that need it, don't make every read pay for it.
- **Ordering hazard:** invalidate-before-commit vs. after matters. `DEL` cache **after** the DB commit succeeds (as shown above), never before — otherwise a concurrent reader can repopulate the cache with the *pre-write* value in the gap between an early delete and the commit, and now the cache is stale until the next write or TTL expiry.

9. Async — invalidation and write-back off the hot path

Interviewer: At 2k writes/sec, is a synchronous `DEL` after every write good enough, or does that need to be async too?

Candidate: The `DEL` itself is fast (memory-speed) so doing it inline after the DB commit is fine at this volume. Where async matters is **fanning invalidation out to other services and cache tiers** that also hold a copy of this data (e.g. a search index, a near-cache in another service, a CDN edge cache of a product page):



This decouples "did my write succeed" (fast, synchronous, just the DB + own cache) from "has every downstream copy of this data heard about it yet" (async, eventually consistent, each consumer catches up at its own pace) — the same async-off-the-critical-path principle as the burger-giveaway and flash-sale docs, applied to cache fan-out instead of an order pipeline.

10. Near-cache — an in-process layer in front of the distributed cache

Interviewer: Even a sub-millisecond Redis call is a network hop. Can we go faster for the hottest keys?

Candidate: Yes — add a small **in-process ("near") cache** inside each app server, holding the tiniest, hottest slice of data (a few thousand keys, short TTL — seconds, not minutes):

```
[ App server ]
├─ 1. check IN-PROCESS near-cache (a few MB, in-memory map, ~microseconds)
│   hit -> return, ZERO network hops
└─ 2. miss -> GET distributed cache (Redis, sub-ms, one hop)
    hit -> populate near-cache, return
    miss -> DB, populate both layers
```

- This is the read-side mirror of the hot-key problem (walked through in §15): if one key gets disproportionate traffic, a near-cache means most of that traffic never even leaves the app server's process, so it never reaches — and can't overload — the one Redis shard that owns that key.
- **The cost:** near-cache copies are per-instance and can be more stale than the shared tier (each app server invalidates on its own schedule/message delivery). Keep the near-cache TTL short and treat it strictly as a latency optimization for the hottest keys, never as the place invalidation correctness lives — that's still the shared cache + TTL + pub-sub above.

11. Technology choices — what and why

Concern	Choice	Why this, and why not the alternative
Cache engine	Redis Cluster	Rich data structures (strings, hashes, sorted sets), built-in replication, cluster-mode sharding, and configurable eviction (<code>maxmemory-policy</code>). <i>Not Memcached</i> — Memcached is a simpler, pure LRU key-value cache with no built-in replication or persistence; fine for pure ephemeral caching, but we want LFU eviction, replicas for failover, and richer structures for things like leaderboard-style sorted sets elsewhere in the product.
Sharding	Redis Cluster hash slots (consistent-hashing family)	Adding/removing a node moves only the slots that node owned, not the whole keyspace — see doc 11 §4 for the general mechanism; we don't re-derive it here. <i>Not $hash(key) \% N$</i> — same mass-reshuffle failure as any store, but worse for a cache: a full reshuffle means a full cold-cache event, i.e. a self-inflicted stampede on the DB.
Eviction policy	<code>allkeys-lfu</code> with decay, plus per-key TTL	Resists scan-pollution (one-off bulk reads don't evict the real hot set) and independently bounds staleness via TTL. <i>Not plain LRU</i> — vulnerable to one large sequential scan evicting the actual working set; <i>not TTL-only</i> — doesn't relieve memory pressure between expirations.
Caching pattern	Cache-aside with invalidate-on-write	Cache only holds what's actually requested (no wasted memory on cold data); a cache outage degrades to "slower reads," never "lost writes," since the DB is always authoritative. <i>Not write-through</i> — adds DB-write latency to every write for a workload that's 1000:1 read-skewed, buying us nothing since writes are rare; <i>not write-back</i> — risks losing product-data writes on a cache-node crash, unacceptable outside of loosely-consistent counters.

Concern	Choice	Why this, and why not the alternative
Replication	Async primary-replica per shard, automatic failover	A replica losing the last few ms of writes is "serve a slightly stale value," not a lost transaction — acceptable for a cache backed by a DB source of truth. <i>Not sync replication</i> — adds a round-trip to every write for a guarantee we don't need; that tradeoff <i>would</i> flip for a system where the cache IS the source of truth (i.e., doc 11's KV store).
Invalidation fan-out	Pub-sub / Kafka event on write, async consumers purge their own copies	Decouples "my write succeeded" from "every downstream cache heard about it," so slow consumers never block the write path. <i>Not synchronous multi-cache invalidation</i> — waiting for N other services to ack a purge before returning from a write turns every write into a distributed transaction.
Hot-key mitigation	Near-cache (in-process) + request coalescing (see §15)	Removes the hottest traffic from the network entirely for the read side; coalescing removes duplicate concurrent misses for the same key. <i>Not "just add more replicas of that one shard"</i> — helps some, but doesn't fix a stampede at expiry (§15) the way coalescing does.
Routing	Cluster-aware client library caching the slot map	One network hop in the common case, like client-side sharding, without every service hand-rolling ring/slot logic; falls back to MOVED / ASK redirects during resharding.

Say-it-out-loud justification: *"Redis Cluster shards with the same consistent-hashing family as any distributed store, but everything downstream of that is cache-specific: LFU eviction with a TTL safety net bounds both memory and staleness; cache-aside with invalidate-on-write keeps the database as the single source of truth so a cache outage degrades gracefully instead of losing data; async replication is fine because the worst case is a stale read, not a lost write; and a small near-cache plus request coalescing protects any single hot key from becoming a single point of overload."*

12. Failure modes & graceful degradation

Failure	What happens	Mitigation
One cache node dies	Its key range goes cold instantly	Replica promotes in seconds (Sentinel/Cluster failover); meanwhile misses fall through to DB, contained to that shard's keys
Entire cache tier down	Every read would hit the DB — the exact outage we built this to prevent	Circuit breaker: app detects cache-tier failure and applies request-level rate limiting / serves slightly-stale-but-cached-at-edge data / sheds non-critical reads rather than let 100% of traffic slam Postgres
Invalidation message lost (pub-sub drop)	A downstream cache serves stale data indefinitely	TTL is the backstop — every key expires eventually regardless of whether invalidation delivery succeeded
Hot key overloads its shard	That one Redis node's CPU/network saturates, its whole slot range degrades for everyone	Near-cache absorbs read traffic before it reaches Redis; replicate the hot key to multiple shards if it's a known always-hot item (see §15)

Failure	What happens	Mitigation
Cache stampede on a popular key's expiry	Thousands of concurrent misses hit the DB at once for the same key	Single-flight/locking or probabilistic early expiration (see §15) — never let N concurrent misses become N concurrent DB queries
DB is down but cache is warm	Reads still succeed from cache (stale-while-revalidate); writes fail	Serve reads from cache with extended/ignored TTL as a degraded mode; queue or reject writes explicitly rather than silently drop them
Cluster resharding in progress	A slot's keys are mid-move; some requests get <code>MOVED</code> / <code>ASK</code> redirects	Cluster-aware clients handle redirects transparently; brief latency bump, no correctness issue

13. Follow-up curveballs

Interviewer: Marketing wants a "live inventory count" on every product page. Cache it?

Candidate: Not the same way as the product description. A count that changes on every order needs either a very short TTL (a few seconds — "close enough" is fine for display) or a push-based update (invalidate on every decrement) if it must be closer to real-time. I'd default to a 2-5 second TTL, cache-aside, same as everything else — a display number tolerating a few seconds of staleness is a much cheaper problem than the actual inventory-decrement correctness, which lives in the database/reservation system, not the display cache.

Interviewer: How do you warm the cache after a full cluster restart, so you don't start cold and cause a stampede on your own DB?

Candidate: Two options depending on urgency: (1) let it warm naturally via cache-aside misses, but **ramp traffic gradually** (a canary/rolling restart, not all nodes at once) so misses arrive as a trickle, not a wall; (2) for known-critical hot keys, run a **warm-up job** that pre-populates the top-N keys (by historical frequency) before the cluster takes real traffic — the same idea as the near-cache's "protect the hottest keys" but applied at cold-start instead of steady-state.

Interviewer: Two app servers both get a cache miss for the same key at nearly the same moment, both query the DB, both write the result back to the cache. Anything wrong?

Candidate: Functionally no — both writes have the same (correct, current) value, so the cache ends up right either way; it's just duplicated DB work for one instant, not a correctness bug. It becomes a real problem when it's not two requests but ten thousand, which is exactly the stampede scenario in §15 — the fix there isn't about correctness, it's about not letting that duplicated work multiply the DB's load by orders of magnitude.

Interviewer: Should the cache ever be allowed to hold data that's *never* in the DB — i.e., is the

source of truth ever the cache itself?

Candidate: For this catalog use case, no — that's the line between "cache" and "the KV-store problem in doc 11." The moment losing a key means losing data (not just losing a speed optimization), you're no longer building a cache; you need the durability, replication quorums, and conflict resolution doc 11 covers. Ephemeral things like rate-limiter counters or session tokens sometimes blur this line deliberately (cache-as-source-of-truth for low-stakes, short-lived data) — but that's an explicit, scoped decision, not a default.

14. Deep-dive: "Why not one giant cache node, or just cache locally in each app server?"

Interviewer: All this sharding, replication, and slot-map machinery is a lot of moving parts. Why not just buy one enormous machine with a terabyte of RAM and run a single Redis instance? Or skip the shared cache entirely and let each app server keep its own local in-memory cache?

Candidate: Both are real options, and both are right at *some* scale — the question is whether they're right at *this* scale, and for both the answer is no, for different reasons.

Option 1 — one giant vertically-scaled node

[500 app servers] —2M reads/sec→ [ONE Redis node, 1 TB RAM]

- **Throughput ceiling:** Redis's core command loop is single-threaded (per shard). One instance tops out around 100-200k ops/sec regardless of how much RAM or CPU you throw at the box — vertical scaling buys you *capacity* (more keys fit) but not *throughput* (ops/sec barely moves past a point). We need ~2M ops/sec; one node gets us a tenth of the way there at best.
- **Blast radius:** every one of those 500 app servers depends on ONE machine. It reboots for a kernel patch, has a bad NIC day, or OOMs — and 100% of read traffic falls through to Postgres simultaneously. A sharded cluster's failure domain is $1/N$ of the keyspace, not all of it.
- **No route to horizontal growth:** when the catalog outgrows even the biggest single instance available, there's no next step except "start sharding" — which is the design you were trying to avoid, just delayed to a more painful, unplanned moment.

Option 2 — client-side-only local caches, no shared tier

[App server 1: local cache] [App server 2: local cache] ... x500
| independent | independent
▼ ▼
each miss goes STRAIGHT to Postgres, no shared layer to absorb it

- **No shared hit rate.** 500 app servers each maintain their own tiny cache of the same hot products. The *effective* memory available for caching product 42 is however much fits on whichever single

instance happens to serve that request — there's no pooling, so the aggregate hit rate on genuinely hot data is far lower than one shared pool of the same total memory would achieve.

- **No coordinated invalidation.** A write has to notify all 500 servers' local caches (or they simply serve stale data until their own local TTL expires) — the pub-sub fan-out from §9 becomes mandatory and much larger (500 consumers instead of a handful of downstream systems), and there's no shared `DEL` that instantly fixes it for everyone.
- **This IS still useful — as the near-cache from §10, layered ON TOP of a shared tier**, precisely because it's cheap and fast for the tiniest hottest slice. The mistake is using it as the *only* layer, not using it at all.

Comparison table

Concern	One giant node (vertical)	Local-only caches (no shared tier)	Sharded distributed cache (chosen)
Throughput ceiling	Capped by one instance's single-threaded ops/sec (~100-200k/s), regardless of RAM/CPU	Scales with app-server count, but each miss still hits the DB independently	Scales near-linearly with shard count
Blast radius on failure	100% of traffic falls to DB at once	Per-instance only, but no shared safety net either	~1/N of keyspace, contained by replicas
Effective hit rate for a given memory budget	Good (one pool) but memory-bound, not throughput-bound	Poor — memory duplicated/fragmented across instances, no pooling	Good — pooled memory, spread across nodes
Invalidation fan-out cost	Trivial — one node to tell	Expensive — must notify every instance, or accept slow per-instance TTL decay	Moderate — a handful of shards + downstream consumers via pub-sub
Growth path	Hits a wall, then requires a rearchitecture anyway	Doesn't really solve the shared-cache problem at any scale	Add shards, ~1/N of keys move, linear growth
Best used as	A dev/staging setup, or a genuinely small dataset+QPS system	A near-cache LAYER on top of a shared tier (§10), not a replacement for one	The shared tier for anything at real production read scale

Rule of thumb: *Vertical scaling and local-only caching aren't wrong ideas — they're the right answer at small scale, and local caching is even the right answer as a layer at any scale. They stop being right the moment either your ops/sec exceeds one node's single-threaded ceiling, or your failure domain needs to be smaller than "the entire cache." At that point you shard — the same conclusion any distributed store reaches, just arrived at from the throughput/blast-radius angle instead of the storage-volume angle.*

15. Deep-dive: cache stampede (thundering herd) and the hot-key problem

Interviewer: Walk me through the nastiest failure mode directly: a hot key — say, the homepage's featured product — has its TTL expire. In the same instant, thousands of requests are in flight for it. What actually happens, second by second, and how do you stop it?

Candidate: This is the sharpest edge case in caching, and it's really **two related problems wearing the same trenchcoat**: the *stampede* (many requests, one key, triggered by expiry) and the *hot key* (one key, so much steady-state traffic that its single shard saturates even without an expiry event). Let me do the stampede first since it's the more dramatic failure.

The scenario

```
t0      key "product:42" (homepage feature) has TTL expiring NOW.
        Steady state before t0: 50,000 req/sec for this key, ALL served from cache, ~0ms.

t0.000  TTL expires. Key is gone from Redis.

t0.001  Request 1 arrives -> GET product:42 -> MISS
t0.001  Request 2 arrives -> GET product:42 -> MISS   (same instant, different request)
t0.001  Request 3 arrives -> GET product:42 -> MISS
        ... all 50,000 requests in this ~1-second window MISS, because none of them
        know another request is ALREADY fetching the same key.

t0.002  ALL 50,000 requests independently do: SELECT * FROM products WHERE id=42
        -> 50,000 concurrent identical queries slam Postgres for ONE row, all at once.

t0.050  Postgres, built for maybe 20-50k qps TOTAL across the whole catalog, now gets
        50,000 qps for a single row's lock/read path -> connection pool exhausted,
        other completely unrelated queries queue behind it -> the ENTIRE catalog
        service degrades because of ONE key's expiry.

t0.100  Eventually all 50,000 responses come back and re-populate the cache -- but the
        damage (DB overload, elevated latency site-wide) already happened.
```

The hot-key variant (no expiry needed): even without a stampede trigger, if `product:42` alone is legitimately 50,000 req/sec of *steady-state* traffic, and Redis Cluster's hashing put it on exactly one shard, that ONE node absorbs 50,000 ops/sec + all of its normal share of other keys — while the other N-1 shards sit comparatively idle. One key can single-handedly saturate one machine in an otherwise perfectly balanced cluster.

```
Shard 1: product:42 alone = 50,000 ops/sec + its normal ~20,000 ops/sec share = SATURATED
Shard 2..N: normal ~20,000 ops/sec each = fine
```

Sharding balances DATA volume evenly. It does NOT balance TRAFFIC to one specific key -- that key lives on exactly one shard no matter how good the hash function is.

Ranked fixes

1. **Locking / single-flight (mandatory for the stampede case).** The first request to miss acquires a short-lived lock (e.g. `SET lock:product:42 NX EX 5`); it alone queries the DB and repopulates the

cache. Every other concurrent request for the same key either waits briefly and retries the cache read, or is served the (slightly) stale value if one still exists rather than piling onto the DB too.

req1: SET lock:42 NX -> got it -> query DB -> SET cache:42 -> DEL lock:42 -> return
req2..50000: SET lock:42 NX -> already locked -> short sleep+retry GET cache:42, or return stale-if-available, instead of also querying the DB This turns "50,000 concurrent DB queries" into "1 DB query, 49,999 requests that wait a few ms for it." This is the single highest-leverage fix and I'd lead with it.

2. **Probabilistic early expiration (prevents the stampede from ever synchronizing).** Instead of a hard TTL where every reader sees the exact same expiry instant, have reads *probabilistically* treat the key as expired slightly **before** its real TTL, with rising probability as it nears expiry — so a handful of requests refresh it *early and staggered*, well before the herd arrives, and the "real" expiry moment finds an already-fresh key. This is the well-known XFetch/"early recomputation" technique — it desynchronizes the herd instead of managing it after the fact.
3. **Request coalescing at a layer above the cache.** Similar to #1 but implemented as a library/proxy behavior: in-flight identical requests for the same key are collapsed into one upstream call, and all waiters share that one result — this generalizes beyond just "cache miss triggers a DB query" to any expensive, duplicable operation, and is often provided for free by cache-client libraries (e.g. `singleflight` patterns).
4. **Never let popular keys expire abruptly — stale-while-revalidate.** Serve the stale value immediately to all callers while ONE background refresh updates it, rather than treating expiry as "delete now, block until refetched." Callers get a fast (if momentarily stale) response instead of waiting on a lock; correctness cost is a few extra seconds of staleness on an already-tolerant read path.
5. **Hot-key replication / sharding the key itself.** For a *known* always-hot key (the homepage feature, not a surprise viral spike), don't leave it as one Redis key on one shard — replicate it to a small pool of read replicas dedicated to hot keys, or split it into `product:42:copy:{0..k}` and have clients pick a random copy, spreading the 50,000 req/sec across k shards deliberately. Combine this with the **near-cache from §10**: if most app servers keep this specific key in-process with a short TTL, the vast majority of that 50,000 req/sec never reaches Redis at all.

Recommended approach

Lead with single-flight locking (#1) as the non-negotiable baseline — it's what actually prevents the N-simultaneous-DB-queries failure, and it's cheap to add to any cache-aside client. **Layer probabilistic early expiration (#2)** so well-known hot keys rarely even reach the stampede trigger in the first place — prevention beats mitigation. For **known, persistent hot keys** specifically (as opposed to one-off unlucky expiries), add **near-caching (#5)** so the traffic never round-trips to Redis at all, and consider deliberately replicating that one key across shards rather than trusting the hash function to spread load it structurally cannot spread.

What to say out loud: "A stampede happens because every concurrent miss for the same key acts independently — the fix is to make them cooperate: one request repopulates, the rest wait or get served stale, via a short-lived lock or single-flight coalescing. Layering probabilistic early expiration means well-known hot keys rarely hit a synchronized expiry moment at all. The hot-key problem is related but distinct — it's not about expiry, it's that sharding balances data volume, not traffic to one

specific key, so a single key can saturate its one shard no matter how good the hash function is. For genuinely hot keys, the fix is to get the traffic off the shared cache entirely — a near-cache in each app process — or deliberately replicate that one key across several shards instead of trusting the hash to spread load it structurally can't."

Summary — the mental model

A distributed cache is **"trade a small, deliberate amount of staleness for two to three orders of magnitude of speed, without ever becoming the thing that can lose data or take down the database it's protecting."** It shares consistent hashing with any sharded store (doc 11), but everything else is cache-specific: **bounded memory forces an eviction policy** (LFU + TTL, resistant to scan-pollution, with staleness capped independent of memory pressure); **cache-aside with invalidate-on-write** keeps the database as the one source of truth so a cache outage degrades gracefully instead of corrupting anything; **async replication** is fine because the worst failure is a stale read, not a lost write; and the hardest edge cases — **cache stampede and hot keys** — are solved by making concurrent misses cooperate (single-flight locking, early expiration) and by getting a hot key's traffic off the shared tier entirely (near-caches, deliberate replication) rather than hoping the hash function spreads load it structurally cannot spread.